

History of Node Js

Node.js is a powerful JavaScript runtime built on Chrome's V8 engine, primarily used for building scalable network applications. Here's a concise timeline of its development and evolution:

♦ 2009 – Creation of Node.js

- **Creator:** Ryan Dahl, a software engineer.
- **Motivation:** Ryan was frustrated with how traditional web servers (like Apache) handled concurrent connections using threads, which consumed significant memory and resources.
- **Innovation:** Introduced **non-blocking, event-driven I/O** to JavaScript, allowing it to handle thousands of concurrent connections efficiently.
- **Built on:** Google's **V8 JavaScript engine**, which compiles JavaScript to native machine code.
- **First release:** May 27, 2009.

♦ 2010 – npm is Introduced

- **npm (Node Package Manager)** is created by **Isaac Z. Schlueter**.
- Simplifies the sharing and reuse of code via packages/modules.
- Quickly becomes a central part of the Node.js ecosystem.

♦ 2011–2014 – Rapid Growth

- Companies like **LinkedIn**, **Uber**, and **PayPal** adopt Node.js.



INFINITECH TRAINING INSTITUTE

[9619 404 202 , 9619 404 225]

[Mulund , Vile Parle]

- Node.js becomes popular for real-time apps, REST APIs, and microservices.
- Community grows, npm package count increases exponentially.

♦ 2015 – Node.js Foundation & io.js Merge

- A fork named **io.js** was created due to slow updates in Node.js.
- **Node.js Foundation** (backed by the Linux Foundation) is formed to manage development.
- **io.js and Node.js** merge again with **Node.js v4.0.0**, bringing improvements and faster releases.

♦ 2016–2020 – LTS and Stability

- Introduction of **Long-Term Support (LTS)** versions.
- Focus on stability, performance, and improved tooling (like ES6 support, async/await).

www.php-training.in / www.phpcourse.in

Mulund : 303,304,309 Shree Samarth Plaza , RRT Road, Opp. Mulund Station . Mulund West . Mumbai 400080

Vile Parle : Datapoint computers: vijaya Apt , Ground Floor ,Bajaj Road , Vile parle West, Vile Parle (West) . MUMBAI -56



INFINITECH TRAINING INSTITUTE

[9619 404 202 , 9619 404 225]

[Mulund , Vile Parle]

- Wider adoption in enterprises and cloud platforms (e.g., AWS Lambda supports Node.js).

♦ 2021–Present – Modern Node.js

- Native **ECMAScript modules (ESM)** support added.
- Enhanced **diagnostics, security, and performance tools**.
- Ongoing development with **Node.js v20+**, regular releases, and community involvement.
- Part of the **OpenJS Foundation** (formed by merging Node.js and JS Foundation).

About Ryan Dahl

- **Full Name:** Ryan Dahl

www.php-training.in / www.phpcourse.in

Mulund : 303,304,309 Shree Samarth Plaza , RRT Road, Opp. Mulund Station . Mulund West . Mumbai 400080

Vile Parle : Datapoint computers: vijaya Apt , Ground Floor ,Bajaj Road , Vile parle West, Vile Parle (West) . MUMBAI -56



INFINITECH TRAINING INSTITUTE

[9619 404 202 , 9619 404 225]

[Mulund , Vile Parle]

- **Background:** Software engineer with experience in mathematics and computer science.
- **Creation Year:** 2009
- **Country:** United States
- **Known For:** Introducing a revolutionary approach to server-side programming with Node.js

Browser JavaScript vs. Node.js

Feature	 Browser JavaScript	 Node.js
Purpose	To make web pages interactive	To build server-side and backend applications
Environment	Runs in web browsers (Chrome, Firefox, etc.)	Runs in a Node.js runtime (outside browser)

www.php-training.in / www.phpcourse.in

Mulund : 303,304,309 Shree Samarth Plaza , RRT Road, Opp. Mulund Station . Mulund West . Mumbai 400080

Vile Parle : Datapoint computers: vijaya Apt , Ground Floor ,Bajaj Road , Vile parle West, Vile Parle (West) . MUMBAI -56



INFINITECH TRAINING INSTITUTE

[9619 404 202 , 9619 404 225]

[Mulund , Vile Parle]

Access to DOM	✓ Yes	✗ No (no window or document object)
File System Access	✗ No (for security reasons)	✓ Yes (<code>fs</code> module for reading/writing files)
HTTP Server	✗ Not built-in	✓ Built-in (<code>http</code> module)
Global Object	<code>window</code>	<code>global</code>
Modules (before ES6)	<code><script></code> tags or AMD (RequireJS)	CommonJS (<code>require</code> , <code>module.exports</code>)
ES Modules (modern)	✓ Yes (<code>import</code> , <code>export</code>)	✓ Yes (with <code>.mjs</code> or <code>"type": "module"</code> in <code>package.json</code>)
APIs	Web APIs: DOM, fetch, alert, etc.	Node APIs: fs, path, os, events, etc.
Security	Sandboxed (limited access for safety)	Full system access (more powerful, less restricted)

www.php-training.in / www.phpcourse.in

Mulund : 303,304,309 Shree Samarth Plaza , RRT Road, Opp. Mulund Station . Mulund West . Mumbai 400080

Vile Parle : Datapoint computers: vijaya Apt , Ground Floor ,Bajaj Road , Vile parle West, Vile Parle (West) . MUMBAI -56



INFINITECH TRAINING INSTITUTE

[9619 404 202 , 9619 404 225]

[Mulund , Vile Parle]

Package Manager	☒ No built-in (use CDNs or bundlers)	☒ Yes (npm , yarn)
Use Case	Frontend (web pages, UI interactions)	Backend (APIs, databases, servers, CLI tools)

Summary

Use Browser JavaScript when:

- You're building UI, web pages, and frontend logic
- You need access to DOM, forms, or user interaction

Npm

npm stands for Node Package Manager.

www.php-training.in / www.phpcourse.in

Mulund : 303,304,309 Shree Samarth Plaza , RRT Road, Opp. Mulund Station . Mulund West . Mumbai 400080

Vile Parle : Datapoint computers: vijaya Apt , Ground Floor ,Bajaj Road , Vile parle West, Vile Parle (West) . MUMBAI -56

It is the default package manager for Node.js and plays a central role in managing JavaScript libraries and dependencies.

What Does npm Do?

1. Installs Packages (Modules)
 - Example: `npm install express` installs the Express.js web framework.
2. Manages Dependencies
 - Tracks and installs all the libraries your project needs.
3. Shares Code
 - Lets developers publish their own packages for others to use.
4. Runs Scripts
 - Automates tasks like testing, building, and starting servers using `npm run`.

Common npm Commands



INFINITECH TRAINING INSTITUTE

[9619 404 202 , 9619 404 225]

[Mulund , Vile Parle]

Command	Description
<code>npm init</code>	Create a package.json file
<code>npm install <package></code>	Install a package locally
<code>npm install -g <package></code>	Install a package globally
<code>npm uninstall <package></code>	Remove a package
<code>npm update</code>	Update installed packages
<code>npm run <script></code>	Run custom scripts from package.json

How to Run Node.js Code

Running Node.js code is simple once you have **Node.js installed** on your system. Here's a step-by-step guide:

Step 1: Install Node.js

- Download and install from: <https://nodejs.org>
- After installation, check the version:
- `node -v`
- `npm -v`

Step 2: Create a JavaScript File

Create a file named, for example, `app.js`:

```
console.log("Hello from Node.js!");
```

www.php-training.in / www.phpcourse.in

Mulund : 303,304,309 Shree Samarth Plaza , RRT Road, Opp. Mulund Station . Mulund West . Mumbai 400080

Vile Parle : Datapoint computers: vijaya Apt , Ground Floor ,Bajaj Road , Vile parle West, Vile Parle (West) . MUMBAI -56



INFINITECH TRAINING INSTITUTE

[9619 404 202 , 9619 404 225]

[Mulund , Vile Parle]

► Step 3: Run the File Using Node

Open your terminal (Command Prompt, PowerShell, or Terminal) and run:

```
node app.js
Output:
Hello from Node.js!
```

Tip: Use **nodemon** (Optional Dev Tool)

- Automatically restarts your app when files change:

```
npm install -g nodemon
nodemon app.js
```

Node js Modules

www.php-training.in / www.phpcourse.in

Mulund : 303,304,309 Shree Samarth Plaza , RRT Road, Opp. Mulund Station . Mulund West . Mumbai 400080

Vile Parle : Datapoint computers: vijaya Apt , Ground Floor ,Bajaj Road , Vile parle West, Vile Parle (West) . MUMBAI -56



In **Node.js**, a **module** is a reusable block of code whose existence does not accidentally impact other code. Modules help organize code into smaller, manageable pieces.

♦ Types of Modules in Node.js

Type	Description
Built-in Modules	Core modules provided by Node.js (no install needed)
Local Modules	Custom modules you create in your project
Third-party Modules	Modules installed using npm (like express, lodash)

✓ 1. Built-in Modules (Core Modules)

Some examples:

- **fs** (File System)
- **http** (Server creation)
- **path** (Working with file and directory paths)
- **os** (Info about operating system)

- `events`, `crypto`, `url`, etc.

✓ Built In Module `fs`

The `fs` (File System) module in Node.js allows you to interact with the file system — to read, write, update, and delete files.

♦ 1. `fs.readFile()` – Asynchronous

```
const fs = require('fs');

fs.readFile('example.txt', 'utf-8', (err, data) => {
  if (err) {
    console.error('Error reading file:', err);
    return;
  }
  console.log('File content:', data);
});
```

♦ 2. `fs.readFileSync()` – Synchronous

```
const fs = require('fs');

try {
  const data = fs.readFileSync('example.txt', 'utf-8');
  console.log('File content:', data);
} catch (err) {
  console.error('Error reading file:', err);
}
```



INFINITECH TRAINING INSTITUTE

[9619 404 202 , 9619 404 225]

[Mulund , Vile Parle]

Key Differences

Feature	readFile() (Async)	readFileSync() (Sync)
Blocking	 Non-blocking	 Blocking
Callback required	 Yes	 No
Performance	Better for large apps	Fine for small/simple scripts
Use case	Servers, apps with concurrency	Simple scripts, setup files

Example File (example.txt)

```
Hello from the file system!
```

Built In Module http

The **http** module is a **built-in module** that allows Node.js to transfer data over the **HyperText Transfer Protocol (HTTP)**.

1.Basic HTTP Server Example

```
const http = require('http');  
  
const server = http.createServer((req, res) => {
```

www.php-training.in / www.phpcourse.in

Mulund : 303,304,309 Shree Samarth Plaza , RRT Road, Opp. Mulund Station . Mulund West . Mumbai 400080

Vile Parle : Datapoint computers: vijaya Apt , Ground Floor ,Bajaj Road , Vile parle West, Vile Parle (West) . MUMBAI -56



INFINITECH TRAINING INSTITUTE

[9619 404 202 , 9619 404 225]

[Mulund , Vile Parle]

```
// Set the response header
res.writeHead(200, { 'Content-Type': 'text/plain' });

// Send response body
res.end('Hello, this is a basic Node.js server!');
});

// Start the server on port 3000
server.listen(3000, () => {
  console.log('Server is running at http://localhost:3000');
});
```

► Run it:

```
node server.js or nodemon server.js
```

 Output:

Open your browser and visit:

```
http://localhost:3000
```

✓ 2. Handle Different URLs

```
const http = require('http');
```

www.php-training.in / www.phpcourse.in

Mulund : 303,304,309 Shree Samarth Plaza , RRT Road, Opp. Mulund Station . Mulund West . Mumbai 400080

Vile Parle : Datapoint computers: vijaya Apt , Ground Floor ,Bajaj Road , Vile parle West, Vile Parle (West) . MUMBAI -56



```
const server = http.createServer((req, res) => {
  res.writeHead(200, { 'Content-Type': 'text/html' });

  if (req.url === '/') {
    res.end('<h1>Home Page</h1>');
  } else if (req.url === '/about') {
    res.end('<h1>About Page</h1>');
  } else {
    res.writeHead(404);
    res.end('<h1>404 Not Found</h1>');
  }
});

server.listen(3000, () => {
  console.log('Server is running at http://localhost:3000');
});
```

Parameter	Type	Description
req	IncomingMessage	Contains info about the request (URL, method, headers)
res	ServerResponse	Used to send a response back to the client

✓ Creating a Local Custom Module



File Structure

project/

├─ **app.js**

└─ **myModule.js**

myModule.js

```
function greet(name) {  
  return `Hello, ${name}!`;  
}  
  
module.exports = greet;
```

app.js

```
const greet = require('./myModule');  
console.log(greet('john')); // Output: Hello, john!
```

Using Third-Party Modules (via npm)

Install a module:

```
npm install multer  
npm install express  
npm install body-parser
```



INFINITECH TRAINING INSTITUTE

[9619 404 202 , 9619 404 225]

[Mulund , Vile Parle]

```
npm install mongoose
```

Use in your file:

```
const multer= require(multer);  
const express= require(express);  
const bodyParser= require('body-parser');  
const mongoose= require(mongoose);
```

Event Loop in Node.js

The **event loop** is a core part of Node.js that allows it to perform **non-blocking I/O operations** — even though JavaScript is **single-threaded**.

It is what makes Node.js highly **efficient**, especially for **I/O-heavy applications** like web servers, APIs, and real-time apps.

Why Is It Important?

Node.js uses **asynchronous callbacks** and the **event loop** to handle multiple operations **without blocking** the main thread

Event Loop Flow (Simplified)

www.php-training.in / www.phpcourse.in

Mulund : 303,304,309 Shree Samarth Plaza , RRT Road, Opp. Mulund Station . Mulund West . Mumbai 400080

Vile Parle : Datapoint computers: vijaya Apt , Ground Floor ,Bajaj Road , Vile parle West, Vile Parle (West) . MUMBAI -56

1. Execute global code
2. Start event loop:
 - Process timers (setTimeout/setInterval)
 - Handle I/O callbacks
 - Process setImmediate
 - Handle close events
 - Execute microtasks (Promises, process.nextTick)
3. Repeat

■ Example: Using setTimeout and setImmediate

```
console.log('Start');

setTimeout(() => {
  console.log('Inside setTimeout');
}, 0);

setImmediate(() => {
  console.log('Inside setImmediate');
});

Promise.resolve().then(() => {
  console.log('Inside Promise');
});

process.nextTick(() => {
  console.log('Inside process.nextTick');
});
```



INFINITECH TRAINING INSTITUTE

[9619 404 202 , 9619 404 225]

[Mulund , Vile Parle]

```
console.log('End');
```

 Output (most likely):

```
Start
End
Inside process.nextTick
Inside Promise
Inside setTimeout
Inside setImmediate
```

REPL in Node.js

REPL stands for:

Read – **E**val – **P**rint – **L**oop

It's an interactive command-line environment that comes built-in with Node.js, allowing you to **run JavaScript code line-by-line**, test snippets, and explore Node.js features quickly.

 REPL Examples

```
> 2 + 2
4

> const name = "john"
undefined
```

www.php-training.in / www.phpcourse.in

Mulund : 303,304,309 Shree Samarth Plaza , RRT Road, Opp. Mulund Station . Mulund West . Mumbai 400080

Vile Parle : Datapoint computers: vijaya Apt , Ground Floor ,Bajaj Road , Vile parle West, Vile Parle (West) . MUMBAI -56



```
> name
'john'

> console.log("Hello from Node.js REPL!")
Hello from Node.js REPL!
undefined
```

REPL Workflow

Read	Takes your input (JavaScript code)
Eval	Evaluates the input
Print	Prints the result
Loop	Waits for the next input

Event Emitter in Node.js

The **EventEmitter** is a **core module** in Node.js that enables communication between objects via **events**.

It forms the **foundation of event-driven programming** in Node.js — just like how you listen to events like clicks in the browser, you can use EventEmitter to listen and react to custom events in Node.js.

Importing EventEmitter

```
const EventEmitter = require('events');
```

```
const myEmitter = new EventEmitter();
```

Basic Example

```
const EventEmitter = require('events');

// Create an instance
const myEmitter = new EventEmitter();

// Register an event listener
myEmitter.on('greet', (name) => {
  console.log(`Hello, ${name}!`);
});

// Emit (trigger) the event
myEmitter.emit('greet', 'john');
```

 Output: **Hello, john!**

🌟 **once()** Example

```
myEmitter.once('shout', () => {
  console.log('This will appear only once');
});

myEmitter.emit('shout');
myEmitter.emit('shout'); // Nothing happens
```

Real-World Use: EventEmitter with HTTP Server

```
const http = require('http');
const EventEmitter = require('events');

const myEmitter = new EventEmitter();

myEmitter.on('requestReceived', () => {
  console.log('Request was received by the server.');
```

```
});

const server = http.createServer((req, res) => {
  myEmitter.emit('requestReceived');
  res.end('Hello World');
});

server.listen(3000, () => {
  console.log('Server running on http://localhost:3000');
```

```
});
```

Why Use EventEmitter?

- Makes your code **modular** and **decoupled**
- Great for **custom events, logging, notifications, data streams**, etc.
- Under the hood, Node.js modules like **http, fs, net, stream**, etc., use EventEmitter.



Express Js in Node.js

Express.js is a fast, minimalist **web framework for Node.js** that simplifies building web servers and APIs.

Think of Express as a layer on top of Node's **http** module — making it easier to handle **routes**, **requests**, **middleware**, and more.

Why Use Express.js?

- ✓ Simple syntax
- ✓ Powerful routing
- ✓ Middleware support
- ✓ Great for building RESTful APIs
- ✓ Widely used in production apps

Step-by-Step: Express.js Setup

✓ 1. Initialize Project

```
mkdir myapp
cd myapp
npm init -y
```

✓ 2. Install Express

```
npm install express
```

3. Basic Express Server

```
// app.js
const express = require('express');
const app = express();

// Route for home page
app.get('/', (req, res) => {
  res.send('Hello from Express!');
});

// Start the server
app.listen(3000, () => {
  console.log('Server running at http://localhost:3000');
});
```

► Run:

```
node app.js
```

Req & res object in express js

In Express.js, **req** and **res** are two key objects used in handling HTTP requests and responses.

They stand for:

- **req** → Request object
- **res** → Response object

These are passed as arguments to route handler functions and give you full control over reading data from the client (**req**) and sending data back (**res**).

Example

```
const express = require('express');
const app = express();

app.get('/hello', (req, res) => {
  res.send('Hello World!');
});

app.listen(3000);
```


req (Request Object)

The **req** object contains information about the HTTP request.

Common Properties:

Property	Description
<code>req.url</code>	The URL of the request
<code>req.method</code>	HTTP method (GET, POST, etc.)
<code>req.headers</code>	The headers sent with the request
<code>req.query</code>	Object containing query string parameters
<code>req.params</code>	Route parameters (e.g., <code>/user/:id</code>)
<code>req.body</code>	Parsed request body (requires <code>express.json()</code>)

res (Response Object)

The **res** object is used to send a response back to the client.

Common Methods:

Method	Description
<code>res.send()</code>	Sends a response (text, HTML, object, etc.)
<code>res.json()</code>	Sends a JSON response
<code>res.status()</code>	Sets the HTTP status code
<code>res.redirect()</code>	Redirects the request to another URL
<code>res.render()</code>	Renders a template (used with view engines)

Package.json

The `package.json` file is the core file of a Node.js or JavaScript project that uses npm (Node Package Manager). It contains important metadata about your project as well as the list of dependencies and scripts used in it.

Example `package.json` File

```
{
  "name": "my-app",
  "version": "1.0.0",
  "description": "A simple Node.js project",
  "main": "index.js",
  "scripts": {
    "start": "node index.js",
    "test": "echo \"No tests specified\" && exit 0"
  },
  "author": "Your Name",
  "license": "MIT",
  "dependencies": {
    "express": "^4.18.2"
  },
  "devDependencies": {
    "nodemon": "^3.0.1"
  }
}
```

Explanation of Fields

www.php-training.in / www.phpcourse.in

Mulund : 303,304,309 Shree Samarth Plaza , RRT Road, Opp. Mulund Station . Mulund West . Mumbai 400080

Vile Parle : Datapoint computers: vijaya Apt , Ground Floor ,Bajaj Road , Vile parle West, Vile Parle (West) . MUMBAI -56

Field	Description
<code>name</code>	The project name (lowercase, no spaces).
<code>version</code>	The current version of the project.
<code>description</code>	A short description of the project.
<code>main</code>	The entry point file (like <code>index.js</code>).
<code>scripts</code>	Custom scripts that can be run with <code>npm run script-name</code> .
<code>author</code>	Your name or organization.
<code>license</code>	Type of license (e.g., MIT, ISC).
<code>dependencies</code>	Libraries needed for the app to run .
<code>devDependencies</code>	Libraries needed only for development (e.g., nodemon).

Common Commands

- `npm init` – Create a **package.json** file interactively.
- `npm init -y` – Create it quickly with default values.
- `npm install express` – Adds **express** to **dependencies**.
- `npm install --save-dev nodemon` – Adds **nodemon** to **devDependencies**.

Package-lock.json

The **package-lock.json** file is an automatically generated file in Node.js projects that use npm (Node Package Manager). It records the exact versions of every

installed package and its dependencies. This ensures consistent installations across different environments.

Purpose of **package-lock.json**:

✓ Locks dependency versions – Prevents accidental upgrades by specifying the exact version of each dependency (and their dependencies).

📦 Improves performance – Speeds up future installs by avoiding version resolution again.

🔒 Ensures consistency – Guarantees that anyone installing the project will get the same versions of packages.

package.json vs **package-lock.json**

Feature	package.json	package-lock.json
Purpose	Declares dependencies	Locks exact dependency versions
Written by	Developer	Automatically generated by npm
Required for install	Yes	Optional (but strongly recommended)
Tracks nested deps	No	Yes
Editable	Yes	No (you should not manually edit it)

Node_modules folder



The `node_modules` folder is where npm (Node Package Manager) stores all the installed packages and their dependencies for a Node.js project.

What is `node_modules`?

It is an **auto-generated directory** created when you run:

```
npm install
```

It contains:

- The dependencies listed in your `package.json`
- All nested dependencies required by those packages

 Structure Example:

```
node_modules/  
├── express/  
├── accepts/  
├── body-parser/  
├── debug/  
└── ...
```

Should I upload `node_modules` to GitHub?

✗ No. Never.

- It's too large and redundant.
- Instead, push only `package.json` and `package-lock.json`.
- Others can run `npm install` to get the same `node_modules`.

Step-by-Step: CRUD API in Express.js

✓ 1. Initialize and Install Express

```
mkdir crud-api
cd crud-api
npm init -y
npm install express
```

✓ 2. Code: `app.js`

```
const express = require('express');
const app = express();
const PORT = 3000;

// Middleware to parse JSON
app.use(express.json());

// Dummy in-memory data
let users = [
```

```
{ id: 1, name: 'john' },
{ id: 2, name: 'Alex' }
];

// ✓ CREATE - Add a new user
app.post('/users', (req, res) => {
  const newUser = {
    id: users.length + 1,
    name: req.body.name
  };
  users.push(newUser);
  res.status(201).json(newUser);
});

// ✓ READ - Get all users
app.get('/users', (req, res) => {
  res.json(users);
});

// ✓ READ - Get a single user by ID
app.get('/users/:id', (req, res) => {
  const user = users.find(u => u.id == req.params.id);
  if (!user) return res.status(404).json({ message: 'User not found' });
  res.json(user);
});

// ✓ UPDATE - Modify an existing user
app.put('/users/:id', (req, res) => {
  const user = users.find(u => u.id == req.params.id);
  if (!user) return res.status(404).json({ message: 'User not found' });
```

```
    user.name = req.body.name;
    res.json(user);
  });

// ✓ DELETE - Remove a user
app.delete('/users/:id', (req, res) => {
  const index = users.findIndex(u => u.id == req.params.id);
  if (index === -1) return res.status(404).json({ message: 'User not found' });

  const deletedUser = users.splice(index, 1);
  res.json(deletedUser[0]);
});

// Start server
app.listen(PORT, () => {
  console.log(`Server is running on http://localhost:${PORT}`);
});
```

Step-by-Step: CRUD API with MongoDB

1. Project Setup

```
mkdir express-mongo-crud
cd express-mongo-crud
npm init -y
npm install express mongoose
```




✓ 2. MongoDB Setup

You can:

- Use **MongoDB Atlas** (cloud) 👉 <https://cloud.mongodb.com>
- Or install MongoDB locally

Get your MongoDB **connection string**, like:

```
mongodb://localhost:27017/userdb  
mongodb+srv://<username>:<password>@cluster.mongodb.net/userdb
```

✓ 3. File: app.js

```
const express = require('express');  
const mongoose = require('mongoose');  
const app = express();  
const PORT = 3000;  
  
// Middleware  
app.use(express.json());  
  
// ✓ Connect to MongoDB  
mongoose.connect('mongodb://localhost:27017/userdb', {  
  useNewUrlParser: true,
```

```
useUnifiedTopology: true
}).then(() => console.log("MongoDB Connected"))
.catch(err => console.log(err));

// ✓ User Schema & Model
const userSchema = new mongoose.Schema({
  name: { type: String, required: true }
});

const User = mongoose.model('User', userSchema);

// ✓ CREATE
app.post('/users', async (req, res) => {
  try {
    const user = new User({ name: req.body.name });
    await user.save();
    res.status(201).json(user);
  } catch (err) {
    res.status(400).json({ error: err.message });
  }
});

// ✓ READ All
app.get('/users', async (req, res) => {
  const users = await User.find();
  res.json(users);
});

// ✓ READ One
app.get('/users/:id', async (req, res) => {
```

```
try {
  const user = await User.findById(req.params.id);
  if (!user) return res.status(404).json({ message: 'User not found' });
  res.json(user);
} catch (err) {
  res.status(400).json({ error: 'Invalid ID' });
}
});

// ✓ UPDATE
app.put('/users/:id', async (req, res) => {
  try {
    const user = await User.findByIdAndUpdate(
      req.params.id,
      { name: req.body.name },
      { new: true }
    );
    if (!user) return res.status(404).json({ message: 'User not found' });
    res.json(user);
  } catch (err) {
    res.status(400).json({ error: err.message });
  }
});

// ✓ DELETE
app.delete('/users/:id', async (req, res) => {
  try {
    const user = await User.findByIdAndDelete(req.params.id);
    if (!user) return res.status(404).json({ message: 'User not found' });
    res.json({ message: 'User deleted', user });
  }
});
```

```
    } catch (err) {  
      res.status(400).json({ error: err.message });  
    }  
  });  
  
//  Start server  
app.listen(PORT, () => {  
  console.log(`Server running at http://localhost:${PORT}`);  
});
```

Test Your MongoDB-Based API

Use **Postman**, **curl**, or **Thunder Client** in VS Code to test:

- GET /users
- POST /users with body: { "name": "john" }
- GET /users/:id
- PUT /users/:id with updated name
- DELETE /users/:id



Template Engine in Node js / Express Js

A template engine allows you to generate dynamic HTML content on the server using data and templates. One of the most popular template engines in Node.js is EJS (Embedded JavaScript).

Why Use a Template Engine?

- To render **dynamic content** in HTML pages.
- To **embed JavaScript** directly inside HTML
- To **separate concerns** (logic vs. presentation).

 Step-by-Step: Node.js with EJS

```
mkdir ejs-example
cd ejs-example
npm init -y
npm install express ejs
```

Project Structure

```
ejs-example/
├── views/
│   └── index.ejs
```



```
| app.js  
| package.json
```

3 Create **app.js**

```
const express = require('express');  
const app = express();  
  
// Set EJS as the template engine  
app.set('view engine', 'ejs');  
  
// Define a route  
app.get('/', (req, res) => {  
  const user = { name: 'John Doe', age: 25 };  
  res.render('index', { user });  
});  
  
// Start server  
app.listen(3000, () => {  
  console.log('Server running at http://localhost:3000');  
});
```

4 Create the **views/index.ejs** file

```
<!DOCTYPE html>  
<html>
```

```
<head>
  <title>EJS Example</title>
</head>
<body>
  <h1>Hello, <%= user.name %>!</h1>
  <p>You are <%= user.age %> years old.</p>
</body>
</html>
```

- ♦ `<%= ... %>` outputs escaped HTML.
- ♦ `<%- ... %>` outputs unescaped HTML.
- ♦ `<% ... %>` runs JavaScript code (no output).

5 Run the Server

```
node app.js or nodemon app.js
```

Visit <http://localhost:3000>

Output:

```
Hello, John Doe!
You are 25 years old.
```



INFINITECH TRAINING INSTITUTE

[9619 404 202 , 9619 404 225]

[Mulund , Vile Parle]

Summary

Feature	Description
Engine Used	EJS
Setup	<code>app.set('view engine', 'ejs')</code>
Views Folder	Default is /views
File Extension	.ejs
Use Case	Dynamic web pages with server-side data

Mongodb Commands

► Start MongoDB Shell

```
mongosh
```

► Show All Databases

```
show dbs
```

► Create/Use a Database

```
use mydb
```

www.php-training.in / www.phpcourse.in

Mulund : 303,304,309 Shree Samarth Plaza , RRT Road, Opp. Mulund Station . Mulund West . Mumbai 400080

Vile Parle : Datapoint computers: vijaya Apt , Ground Floor ,Bajaj Road , Vile parle West, Vile Parle (West) . MUMBAI -56



► Show Current Database

```
mydb
```

► Show All Collections in Current DB

```
show collections
```

► Create a Collection

```
db.createCollection("users")
```

► Insert One Document

```
db.users.insertOne({ name: "john", age: 25 })
```

► Insert Many Documents

```
db.users.insertMany([  
  { name: "Alex", age: 30 },  
  { name: "Sam", age: 22 }  
]);
```



► Find All Documents

```
db.users.find()
```

► Find with Condition (Query)

```
db.users.find({ age: { $gt: 24 } })
```

► Update One Document

```
db.users.updateOne(  
  { name: "john" }, { $set: { age: 26 } }  
);
```

► Update Many

```
db.users.updateMany(  
  { age: { $lt: 30 } },  
  { $set: { status: "young" } }  
);
```

► Delete One



INFINITECH TRAINING INSTITUTE

[9619 404 202 , 9619 404 225]

[Mulund , Vile Parle]

```
db.users.deleteOne({ name: "Sam" })
```

► Delete Many

```
db.users.deleteMany({ age: { $lt: 25 } })
```

► Drop a Collection

```
db.users.drop()
```

► Drop a Database

```
db.dropDatabase()
```

► Count Documents

```
db.users.countDocuments()
```

► Count Documents

www.php-training.in / www.phpcourse.in

Mulund : 303,304,309 Shree Samarth Plaza , RRT Road, Opp. Mulund Station . Mulund West . Mumbai 400080

Vile Parle : Datapoint computers: vijaya Apt , Ground Floor ,Bajaj Road , Vile parle West, Vile Parle (West) . MUMBAI -56



```
db.users.countDocuments()
```

1. Comparison Operators

\$eq – Equal

```
db.users.find({ age: { $eq: 25 } })  
// Finds users with age exactly 25
```

\$ne – Not Equal

```
db.users.find({ age: { $ne: 30 } })  
// Finds users whose age is not 30
```

\$gt – Greater Than

```
db.users.find({ age: { $gt: 18 } })  
// Users older than 18
```

\$gte – Greater Than or Equal

```
db.users.find({ age: { $gte: 18 } })  
// Users age 18 and above
```



\$lt – Less Than

```
db.users.find({ age: { $lt: 30 } })  
// Users younger than 30
```

\$lte – Less Than or Equal

```
db.users.find({ age: { $lte: 25 } })  
// Users age 25 or younger
```

\$in – Value in Array

```
db.users.find({ name: { $in: ["john", "Alex"] } })  
// Users with name either "john" or "Alex"
```

\$nin – Not in Array

```
db.users.find({ age: { $nin: [18, 21] } })  
// Users whose age is not 18 or 21
```

\$and – Logical Operators



```
db.users.find({
  $and: [{ age: { $gt: 20 } }, { age: { $lt: 30 } }]
})
// Users between age 21-29
```

\$or –Logical Operators

```
db.users.find({
  $or: [{ name: "john" }, { age: 25 }]
})
// Users named "john" OR age 25
```

\$not –Logical Operators

```
db.users.find({
  age: { $not: { $gt: 30 } }
})
// Users NOT older than 30
```

\$nor –Logical Operators

```
db.users.find({
  $nor: [{ name: "Alex" }, { age: 20 }]
})
// Users who are NOT named Alex AND NOT age 20
```



\$set –Update Operators

```
db.users.updateOne({ name: "john" }, { $set: { age: 26 } })  
// Updates age to 26
```

\$unset –Update Operators

```
db.users.updateOne({ name: "Alex" }, { $unset: { age: "" } })  
// Removes the age field
```

\$inc–Update Operators

```
db.users.updateOne({ name: "john" }, { $inc: { age: 1 } })  
// Increments age by 1
```

\$mul–Update Operators

```
db.users.updateOne({ name: "john" }, { $mul: { age: 2 } })  
// Multiplies age by 2
```



\$rename-Update Operators

```
db.users.updateOne({}, { $rename: { "username": "name" } })  
// Renames 'username' field to 'name'
```

\$exists-Element Operators

```
db.users.find({ phone: { $exists: true } })  
// Users who have a phone field
```

\$type-Element Operators

```
db.users.find({ age: { $type: "number" } })  
// Finds users with age as a number
```

\$all-Array Operators

```
db.users.find({ skills: { $all: ["JavaScript", "Node.js"] } })  
// Users who have both skills
```

\$elemMatch-Array Operators

```
db.users.find({  
  scores: { $elemMatch: { $gt: 80, $lt: 100 } }  
})
```




INFINITECH TRAINING INSTITUTE

[9619 404 202 , 9619 404 225]

[Mulund , Vile Parle]

```
// At least one score between 81 and 99
```

\$size-Array Operators

```
db.users.find({ tags: { $size: 3 } })  
// Tags array with exactly 3 items
```